

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

José Ignacio Valenzuela Von Unger

11 de septiembre de 2026

02:06

1. Introducción

En este informe se presenta el análisis experimental de distintos algoritmos clásicos para ordenar arreglos y multiplicar matrices. El objetivo principal es ver cómo se comportan estas implementaciones en la práctica y comparar los tiempos reales obtenidos con su complejidad teórica, para entender cómo afectan los detalles de implementación a la hora de programarlos.

2. Entorno experimental

Los experimentos se corrieron en un notebook ASUS ROG Strix G15 con una tarjeta gráfica NVIDIA GeForce RTX 3070ti y sistema operativo Windows. Los algoritmos se programaron en C++ y se compilaban usando GCC. Para medir la memoria utilizada en cada prueba (*Working Set Size*), se ocupó la librería `psapi.h` de Windows. Por otro lado, los gráficos de los resultados se generaron automáticamente mediante un script en Python utilizando Pandas y Matplotlib. Los datasets evaluados corresponden exactamente a los descritos en el enunciado de la tarea.

3. Resultados y Análisis

3.1. Ordenamiento de Arreglos

Se probaron arreglos de hasta 10^7 elementos. En el siguiente gráfico se pueden ver los tiempos de ejecución registrados.

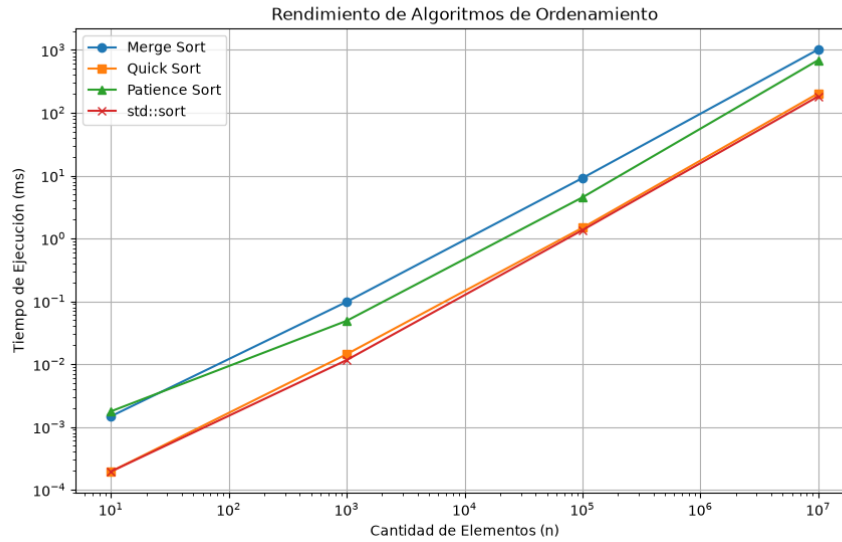


Figura 1: Tiempos de ejecución para algoritmos de ordenamiento.

Algo importante que notamos en las pruebas fue el comportamiento de Quick Sort. Al probarlo con arreglos gigantes y con muchos números repetidos, la versión clásica se fue a su peor caso de $O(n^2)$. Para evitar que el proceso se demorara una eternidad, implementamos una partición de 3 vías. Con eso, los tiempos volvieron a ser de $O(n \log n)$ y el algoritmo logró competir muy de cerca con el `std::sort` nativo de C++.

Adicionalmente, al contrastar estos resultados con los peores casos teóricos de cada algoritmo, el comportamiento varía drásticamente según la naturaleza de los datos. Algoritmos como Merge Sort y `std::sort` (que internamente implementa Introsort) mantienen un rendimiento altamente consistente de $O(n \log n)$ sin importar si el arreglo entra ordenado, invertido o desordenado. Sin embargo, Patience Sort es extremadamente sensible a la distribución inicial: frente a su peor caso (un arreglo inversamente ordenado), el algoritmo se ve forzado a crear n pilas individuales. Esto no solo degrada su tiempo de ejecución por la enorme cantidad de inserciones y fusiones en la cola de prioridad, sino que dispara agresivamente el consumo de memoria en comparación a los algoritmos in-place.

3.2. Multiplicación de Matrices

Para esta segunda parte, comparamos el método clásico de Fuerza Bruta (Naive, $O(n^3)$) con el algoritmo de Strassen ($O(n^{2.81})$).

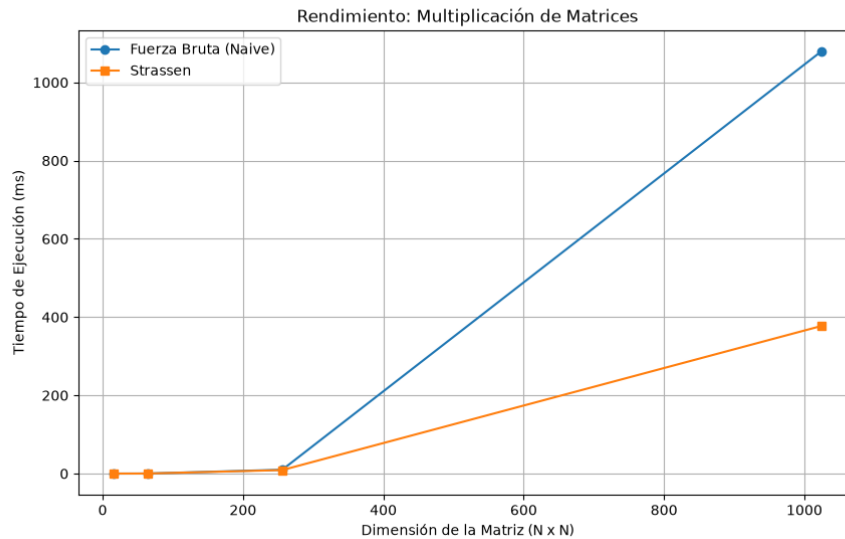


Figura 2: Tiempos de ejecución para multiplicación de matrices.

Como se observa en el gráfico, para matrices pequeñas y medianas ($N \leq 256$), ambos algoritmos registran tiempos muy bajos y visualmente similares. Sin embargo, al evaluar las matrices de 1024×1024 , el costo del algoritmo de Fuerza Bruta se dispara por sobre los 1000 ms, mientras que Strassen escala de manera mucho más eficiente, manteniéndose por debajo de los 400 ms. Esta enorme brecha de rendimiento demuestra en la práctica el impacto de reducir la complejidad asintótica de $O(n^3)$ a $O(n^{2.81})$ al procesar grandes volúmenes de datos.

3.3. Análisis de Consumo de Memoria

Durante las mediciones de consumo de memoria (*Working Set Size*), se registraron ocasionalmente valores delta negativos al finalizar la ejecución de los algoritmos con los archivos más grandes. Es importante destacar que esto no representa un error en el código, sino que evidencia un comportamiento nativo del sistema operativo (Windows). El administrador libera y pagina dinámicamente la memoria RAM hacia el disco duro durante el procesamiento masivo de datos, lo que en ciertas ventanas de tiempo resulta en una lectura final de memoria física asignada menor a la lectura inicial.

4. Implementaciones

Link del repositorio de GitHub:

<https://github.com/Joz3001/INF221-Tarea1>

5. Conclusiones

Hacer este análisis nos sirvió para comprobar que la notación Big-O realmente ayuda a predecir cómo se van a comportar los algoritmos cuando procesan millones de datos. Pero la gran conclusión es que la teoría no lo es todo. En la práctica, las pequeñas decisiones al momento de programar (como usar la partición de 3 vías para evitar los repetidos o frenar la recursividad en Strassen) son las que realmente salvan el código y hacen que los algoritmos funcionen rápido en la vida real.